

CHIA-OpenROAD: An Agentic RTL-to-GDS Loop with a Replaceable Surrogate Evaluator

Barsat Khadka

CHIA Hackathon at the A³ Workshop, MICRO 2026

AI usage

AI was used to draft this write-up and to write the code for the hackathon. The results reported here were produced by running that code and were verified by the author.

Code: <https://github.com/BarsatKhadka/CHIA-OpenROAD>

1 Context and Contributions

CHIA is an open framework from UC Berkeley for building agentic hardware/software co-design flows on a distributed Ray runtime [2]. A workflow is ordinary Python. A *node* is a class whose methods, marked `@ChiaFunction`, run on cluster workers, and a `ChiaTool` exposes a chosen set of capabilities to an agent over MCP. Its published case studies share the pattern of agent proposing and editing, and a trusted, AI-free evaluation decides whether the result is accepted.

This paper contributes a surrogate interface with three unlike implementations behind it, one a published clock-tree model [5], in which a model declares the stage it observes and the artifacts it needs and the loop supplies file paths rather than features. It contributes a trust boundary enforced in code rather than in a prompt, under which no tool the agent can call runs sign-off, parses a report, or invokes the flow. It contributes four measured causes of an agentic tuning loop failing to improve across turns, each fixed and each verified by the behaviour changing rather than the score alone. And it contributes a set of failures that produce plausible numbers rather than errors, which anyone building such a loop will meet.

2 The loop

2.1 Where the flow runs

The unit of execution is a single container image that is simultaneously a complete OpenROAD-flow-scripts (ORFS) [1, 8] installation and a valid CHIA worker. It is built on the upstream ORFS image and adds the Python environment CHIA requires (Ray and chia on Python 3.10) so that a `@ChiaFunction` body runs inside the container and invokes `make` directly, rather than a host process shelling out to `docker run`. Building the image first forces the local and cluster execution models to be identical from the start.

Failure mode: ORFS’s helper scripts import `yaml` and `pandas` from whatever `python3` resolves to, via `PYTHON_EXE` `!= $(shell command -v python3)`. Once CHIA’s conda environment is active, which is how the Ray worker starts, that resolves to conda’s interpreter, which does not have them. Pinning `PYTHON_EXE` to the system interpreter in the image keeps ORFS’s scripts on the interpreter they were built against while CHIA keeps its own.

2.2 Running a stage

Taking RTL to a GDSII is a fixed sequence of physical-design steps, and ORFS packages each as a make target that reads the previous step’s checkpoint and writes its own (Figure 1). A decision taken early constrains everything after it and every step leaves a checkpoint, so a flow can resume from any point rather than repeat. These stages are controlled through knobs: make variables that ORFS’s scripts read as they run. There are 249 of them in the version used here (OpenROAD 26Q3-1817), of which ORFS marks 18 as tunable, and each is read at a particular step, so changing one invalidates that step and every step after it. The flow is wrapped as a node, `OpenROADNode`. Its `run_stage` member takes a work directory, a design configuration and a knob mapping to invoke the corresponding make target inside the container which returns the checkpoint written, the elapsed wall-clock time, and the metrics ORFS records in its per-stage JSON. No value is recovered by parsing log text. A knob reaches the flow as `KNOB_NAME=VALUE` appended to that make command. Every knob is therefore checked against a small set of rules before invocation. A name must be upper-snake-case, since it is placed in an `argv` where an unconstrained key could introduce an additional flag or make target, and it must appear among the 249 known variables (extracted from ORFS’s own `variables.json` into a generated `knob_specs` module), so a misspelling never reaches the flow.

Failure mode: `make lvs` exits 0 whether or not the netlists match. The `sky130hd` deck has its `raise` commented out and merely prints the error, so the return code certifies a layout that has failed LVS as clean. So the verdict of `lvs` must be read from the report and not the exit status.

2.3 Invalidation

While changing a knob must invalidate the step that reads it and every step after, the build system does not implement this. Make decides staleness by comparing file timestamps, and a knob is not a file. This was confirmed with `make -n`, which reports that everything is up to date after `CORE_UTILIZATION` (floorplan) is changed, after `CTS_CLUSTER_SIZE` (clock-tree synthesis) is changed, and after setting a variable that does not exist at all. A search loop built directly on this would request a new configuration and receive the previous design in its place, with nothing in the result to indicate the substitution.

Invalidation is therefore performed explicitly. Each work directory carries a manifest recording the knobs that produced its checkpoints. Before a stage is invoked, the requested knobs are compared against that manifest, the earliest step at which any differing knob is read is determined from the extracted variable table, and every checkpoint from that step onward is deleted. A change confined to clock-tree knobs therefore preserves synthesis, floorplanning and

placement, while a change to core utilisation discards everything from floorplanning on.

2.4 On a cluster

Evaluating one configuration means running a design through the entire flow (Figure 1), and the number of admissible configurations grows combinatorially with the number of knobs, so exhaustive evaluation is infeasible. The knobs also interact, for example placement density must exceed core utilisation or placement cannot legalise, and the effect of a clock-tree knob depends on the floorplan beneath it. Design space exploration therefore reduces to choosing few configurations carefully and evaluating those few in parallel. Ray has no notion of ORFS, so the cluster declares a custom resource, `orfs`, with a count, and each candidate reserves one unit of it for the duration of its build. Once the units are taken, further candidates queue rather than oversubscribe the machine. The thread count handed to each build follows from the same number, namely the machine’s cores divided by the declared unit count. The members of a node cannot be scheduled independently. `run_stage` writes a work directory on a worker’s local disk, and the members that later read metrics or measure the clock tree operate on that same directory, so all of them must execute on the worker that holds it. Declaring the node as a `ColocatedNode` places its members in a single placement group and satisfies this.

3 The trust boundary

The agent participates by proposing configurations, not by executing anything. Each iteration renders the ledger of what has been built, what it was derived from, its measured metrics, and what failed and why, as text. That text is the entire prompt. The agent replies with configurations, the driver validates each one and dispatches the survivors, and the results are written back to the ledger. The agent holds no state between iterations, so its memory is the ledger and nothing else.

Within a turn the agent may call the read-only tools listed in Table 1. All of them are database queries or fitted-model evaluations and complete in milliseconds. Proposing a configuration is not among them, because it is not a tool call at all. The agent’s reply is text, and the driver parses the configurations out of it, validates each one, and dispatches the builds itself. The set of tools is not fixed. A tool is a method on a `ChiaTool`, so adding one is adding a method, and the boundary is preserved by what a tool is permitted to do rather than by keeping the list short. This is where the pattern from CHIA’s case studies is enforced. The agent never runs the flow. It names a configuration and the driver runs it, so every number in the ledger is one ORFS reported rather than one the agent supplied. It cannot run DRC or LVS, override knob validation, or force a rebuild that would bypass invalidation. Whether a configuration builds at all is decided by ORFS, and when one fails the recorded reason is the tool’s own message, for example that global routing finished with congestion.

Failure mode: what matters is not whether the agent may call tools but how long those calls take. An earlier design gave the agent the build as a tool and held one model session open across it. On two occasions the session ignored its timeout and hung for over two hours while the build completed underneath it and went

Table 1: The tools available to the agent within a turn. Every one is read-only and completes in milliseconds. The set is extensible; the constraint is on what a tool may do, not on how many exist.

Tool	Returns
<code>list_legal_knobs</code>	the settable knobs, with stage and range
<code>past_failures</code>	configurations known not to build, and why
<code>list_candidates</code>	every configuration evaluated so far
<code>compare_candidates</code>	those configurations ranked by a chosen metric
<code>best_candidate</code>	the best configuration by a chosen metric
<code>screen_candidates</code>	a precomputed surrogate ranking over a grid
<code>describe_surrogates</code>	what each fast model reads and predicts
<code>predict_knobs</code>	predictions for configurations the agent supplies

uncollected. Five successive runs failed this way, none for reasons connected to the agent’s reasoning. Hence, in this design fast tools belong inside the turn but the build belongs outside it.

4 The surrogate socket

Running a stage is expensive, so surrogate models have started being widely used in the flow.

Every surrogate predicts from features of a completed stage, but which features differ from model to model. We therefore do not compute features. A model declares the stage it observes and the files it needs, the loop produces those files, and the author writes the parsing inside the model. Figure 2 shows the form. The model returns one prediction per configuration. The agent invokes a model and gets a prediction back. Each iteration it also names a shortlist wider than it can afford to build, and the models price that shortlist before it commits. A grid of configurations is scored once in advance, so the agent can read off the best predicted combinations without asking for them. A prediction is trusted only inside the range the model was fitted on, so configurations outside that range are never sent to it.

Failure mode: a fitted model cannot cross into the tool’s MCP actor. The tool is serialised when that actor is constructed and the actor has no import path for a plugin’s dependencies, so the run fails at startup with a missing-module error that names neither the tool nor the model. The driver’s copy holds the models and answers prediction requests; the actor’s copy holds none and says so.

5 Experimental setup

Four designs are taken from CTS-Bench [4] and ported into ORFS on the sky130hd open PDK [7], keeping the clock ports and top modules the benchmark uses. Each is given a clock period at which the shipped configuration lands close to timing closure, so the search has room to work. The periods are 6.5 ns for `aes`, 4.5 ns for `picorv32`, 10.4 ns for `sha256` and 6.5 ns for `ethmac`.

The agent is `gemini-2.5-flash` [3] reached through Vertex AI, at temperature 0.5 when it proposes configurations and 0.4 when it calls tools. No model is fine-tuned, and nothing carries between runs except the ledger that run writes.

All experiments were conducted on a single `n2-standard-32` instance on Google Cloud, 32 vCPUs in `us-west1-b`, running the CHIA container. The project is capped at 32 vCPUs globally, so

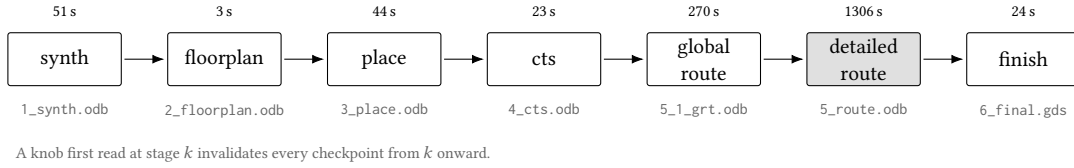


Figure 1: The ORFS stage chain, with the checkpoint each stage leaves and its measured cost on aes/sky130hd at 8 threads (1721 s total). Detailed routing is 76% of the flow, so any knob change that reaches it costs close to a full build.

```

class MyModel(SurrogateEvaluator):
    observes_stage = "place"
    requires      = ("def", "timing_rpt")
    predicts      = {"clock_skew_setup": "orfs_metric"}
    domain        = {"CTS_CLUSTER_SIZE": (12, 30)}

    def _predict(self, state, candidates):
        d = parse(state.artifacts["def"])
        ...
  
```

Figure 2: A surrogate declares the stage it observes and the artifacts it requires. The loop produces those artifacts and passes their paths; parsing and feature extraction happen inside the model.

throughput comes from evaluating candidates concurrently rather than from larger machines. Each build is given two threads, which allows sixteen to run at once.

6 Results

6.1 The four arms

Four arms are compared. *Default* builds the shipped configuration once. *Screen* builds the surrogate’s top-ranked configurations with no LLM. *Agent* runs the loop with the surrogate withheld. *Full* is the whole loop. Each search arm gets twelve builds.

No arm wins everywhere, and the disagreements are sharp (Table 2). On aes the surrogate ranking alone reaches the best slack and clears the default in ten builds of twelve, while the full loop finds a slightly worse optimum using 8.5% less power and 2.4% less area and paying 15% worse skew for it. On ethmac the full loop is best on every metric at once. On sha256 the surrogate ranking fails outright, with no build beating the default, while both LLM arms beat it. On picorv32 nothing works as every arm’s median is worse than the shipped configuration.

Two columns are reported because the conclusion depends on which is read. Ranking arms by their single best result rewards variance rather than quality. On picorv32 the screen arm has the best single candidate while only two of twelve beat the default and its median is worse than doing nothing. On aes the two statistics disagree in the other direction, the full loop holding the best median while placing second on best-of-twelve.

The *preCTS* column counts builds that varied a knob read before clock-tree synthesis. It is zero throughout for the screen arm, because the precomputed ranking covers clock-tree knobs only. That arm differs from the agent arms in two respects at once, the LLM

and the search space, and is not a clean measurement of what the LLM contributes.

6.2 Making the loop improve across turns

The loop above does not learn. Measured over three turns on four designs, the best result of each turn was flat or oscillating in seven of eight runs, with the third turn frequently worse than the second. Three causes were found, all in what the agent is shown rather than in the model.

The prompt required every candidate to differ from everything already built, which makes refining a good configuration illegal and restarts the search each turn. A candidate derived from an earlier one did not inherit its parent’s knobs, so an instruction to change one knob produced a near-default build with one knob set, and every refinement was silently a fresh sample. And the ledger never showed the default the loop was meant to beat, so a run where all twelve candidates lost looked identical to one where all twelve won.

Fixing all three changes the behaviour rather than merely the score. On picorv32 the per-turn best went from -0.2878 , -0.3149 , -0.3459 , monotonically worse, to -0.2801 , -0.2455 , -0.2351 , monotonically better, with the median improving by 0.095 ns. Inheritance is visible directly in the data: the mean number of knobs per candidate rose from 3.0–3.8 to 5.5–7.3.

A fourth change followed from the same diagnosis. The agent carries priors from other designs, and on sha256 they are wrong. Pooling every build of that design shows CORE_UTILIZATION= 45 averaging -0.675 against 30 at -0.864 , the opposite of what helps aes and picorv32, and SETUP_SLACK_MARGIN= 0.0 averaging -0.682 against 0.1 at -0.778 . The agent could not see this from a table of twelve rows with six to eight knobs varying at once, so the loop now computes each knob’s aggregate effect from its own ledger and renders it, noting that the evidence holds for that design only.

6.3 The loop after those changes

Table 3 reports the full loop with all four changes, evaluated multi-fidelity. Every design improves on its default, and on three of the four nearly every routed candidate does. sha256 is the clearest case: the design that produced three wins in 89 builds now produces seven in eight.

6.4 What multi-fidelity costs

Candidates are scored at clock-tree synthesis and only the best are routed, after StateTune [6], which evaluates at a cheap stage and promotes selectively. A quick evaluation averaged 1.0 min against 22.9 min for a full build on picorv32, about 4%, because detailed

Table 2: Four arms on four designs, twelve builds each. *best* is the best worst-slack reached, *med* the median across that arm’s builds, *beat* how many beat the default. *preCTS* counts builds that varied a knob read before clock-tree synthesis; it is zero for the screen arm, whose ranking covers clock-tree knobs only. Slack in ns, power in W, area in μm^2 .

design	arm	best	med	power	area	beat	preCTS
aes	default	-0.2755	-	0.0427	2.53e5	-	-
	screen	-0.1478	-0.2304	0.0420	2.51e5	10/12	0/12
	agent	-0.1651	-0.2621	0.0427	2.53e5	6/11	10/11
	full	-0.1618	-0.1987	0.0390	2.47e5	7/10	9/10
picorv32	default	-0.2352	-	0.0354	1.03e5	-	-
	screen	-0.1660	-0.2643	0.0333	1.02e5	2/12	0/12
	agent	-0.1772	-0.2645	0.0343	1.03e5	1/11	9/11
	full	-0.2878	-0.3723	0.0347	1.03e5	0/12	12/12
sha256	default	-0.5778	-	0.0161	1.25e5	-	-
	screen	-0.6018	-0.6381	0.0148	1.24e5	0/12	0/12
	agent	-0.5471	-0.7001	0.0157	1.37e5	1/10	9/10
	full	-0.5342	-0.7125	0.0216	1.30e5	1/12	11/12
ethmac	default	+0.1106	-	0.1505	4.82e5	-	-
	screen	+0.1396	+0.0659	0.1322	4.68e5	2/12	0/12
	agent	+0.3169	+0.1337	0.1837	5.12e5	7/12	10/12
	full	+0.3703	+0.2437	0.1319	4.67e5	9/12	12/12

Table 3: The full loop after the four changes of §5.2, evaluated multi-fidelity: 24 configurations scored at clock-tree synthesis, the best eight routed to GDS. sha256 is measured at 10.4 ns.

design	default	best	Δ	beat
aes	-0.2755	-0.1651	+0.1104	5/6
picorv32	-0.2352	-0.1874	+0.0478	6/8
sha256	-0.3921	-0.1513	+0.2408	7/8
ethmac	+0.1106	+0.4131	+0.3025	8/8

routing is roughly three quarters of a flow. Exploring 24 configurations and routing eight cost 363 min on sha256 against an estimated 1007 min had all 24 been routed, and 215 min against 566 min on picorv32 – 2.8 $\times$ and 2.6 $\times$ cheaper for the same coverage. The saving is smaller than the per-build ratio suggests because promoted builds still pay full price.

The trade is real. On picorv32 the all-full run reached -0.1482 while the multi-fidelity run reached -0.1874: routing eight of 24 candidates rather than all of them means a good configuration can go unpromoted, since quick-stage slack ranks candidates at a Spearman correlation of +0.77 against final slack rather than perfectly.

7 Limitations

Every cell is a single run. Two configurations that allocate their budget identically differed by 0.057 ns in median on the same design, so that is the noise floor, and differences smaller than it are not claimable. Seeds are the obvious remedy and were not affordable within the compute budget.

The screen arm is not a clean control, as §5.1 notes: it varies both the LLM and the search space. An arm restricting the agent to clock-tree knobs would isolate the LLM as a single variable.

The surrogate is used for one of the three quantities it declares. On these designs it tracks clock wirelength to 2.6–3.1% mean absolute error, but ORFS produces no SAIF, so its power prediction has rank correlation 0.00 and is unusable, and clock skew is not

produced at all. An arm labelled “with a surrogate” that screens one of three declared objectives should say so.

Sign-off is partial. DRC is clean; LVS is blocked by ORFS’s own collateral, where sky130hd.cd1 is in a dialect the packaged KLayout cannot read and the netlists do not match once that is worked around.

References

- [1] Tutu Ajayi and David Blaauw. 2019. OpenROAD: Toward a Self-Driving, Open-Source Digital Layout Implementation Tool Chain. In *Proceedings of the Government Microcircuit Applications and Critical Technology Conference (GOMACTech)*. 1105–1110.
- [2] Angela Cui, Ferran Hermida-Rivera, Jack Toubes, Raghav Gupta, Jim Fang, Chengyi Lux Zhang, Ella Schwarz, Junha Kim, Yakun Sophia Shao, Borivoje Nikolic, Christopher W. Fletcher, and Sagar Karandikar. 2026. CHIA: An Open-Source Framework for Principled, Agentic AI-Driven Hardware/Software Co-Design Research. *arXiv preprint arXiv:2606.27350* (2026). <https://arxiv.org/abs/2606.27350>
- [3] Google DeepMind. 2025. Gemini 2.5 Flash. <https://deepmind.google/models/gemini/flash/>
- [4] Barsat Khadka, Kawsher Roxy, and Md Rubel Ahmed. 2026. CTS-Bench: Benchmarking Graph Coarsening Trade-offs for GNNs in Clock Tree Synthesis. *arXiv preprint arXiv:2602.19330* (2026). <https://arxiv.org/abs/2602.19330>
- [5] Barsat Khadka, Kawsher Roxy, and Md Rubel Ahmed. 2026. SwiftCTS: Fast Cross-Design Prediction and Pareto Optimization of Clock Tree Metrics via Few-Shot Calibration. *arXiv preprint arXiv:2606.11348* (2026). <https://arxiv.org/abs/2606.11348>
- [6] Kunlong Li, Shangshang Yao, Su Zheng, and Lingli Wang. 2026. StateTune: Transforming LLM-Assisted EDA Flow Tuning into a Stateful, Closed-Loop Process. *arXiv preprint arXiv:2608.23601* (2026). <https://arxiv.org/abs/2608.23601>
- [7] SkyWater Technology and Google. 2020. SkyWater Open Source PDK. <https://github.com/google/skywater-pdk>
- [8] The OpenROAD Project. 2026. OpenROAD-flow-scripts. <https://github.com/The-OpenROAD-Project/OpenROAD-flow-scripts>